

COMMON COURSE ASSIGNMENT REPORT

Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	BE
Course Code & Course Name	CSA0904 - Programming in Java
Academic Year / Batch	2024
Faculty Name	Dr. R. Thalapathi Rajasekaran
Assignment Title	Smart Library Management, Book Reservation and Overdue Notification System using Java
Date of Issue	31.08.2026
Date of Submission	02.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations. CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and

	generics to store, retrieve, and process groups of objects in web applications. CO3: Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques.
Bloom's Taxonomy Level	L2 – Understand; L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure; SDG 16 – Peace, Justice and Strong Institutions

Table of contents

S.NO	Contents	Page No
1.	Problem Statement	3-4
2.	Objective	5
3.	Requirements and Environment Used	6
4.	Design / Proposed Solution	7
5.	Algorithm / Pseudocode / Flowchart*, wherever applicable	8-12
6.	Implementation / Source Code	13-16
7.	Test Cases and Expected/Actual Results	17
8.	Execution Screenshots / Output	18-20
9.	Analysis and Discussion	20-24
10.	Conclusion	25
11.	Individual Contribution of Group Members	26-27
12.	References	29
13.	One page Write up	33-35

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

1.1 Problem Statement

Libraries manage a large number of books, members, borrowing transactions, returns, reservations, and overdue records. Performing these activities manually can lead to problems such as inaccurate inventory records, duplicate reservations, difficulty in tracking due dates, and delayed notifications.

The objective of this project is to design and implement a menu-driven Smart Library Management System using Java. The system automates major library operations such as:

- Adding and managing books
- Registering library members
- Searching books
- Issuing books
- Returning books
- Cancelling issued books
- Maintaining available book stock
- Reserving unavailable books
- Managing reservation waitlists
- Calculating overdue fines
- Sending due-date and overdue notifications
- Generating circulation reports
- Generating inventory utilization reports
- Handling invalid operations using exceptions
- Simulating concurrent operations using multithreading

The system uses Object-Oriented Programming principles, Java Collection Framework, exception handling, and multithreading concepts.

1.2 Problem Formulation

The library system can be formulated as a collection of interacting entities.

Main Entities

Entity	Responsibility
Member	Stores member information and borrowing details
Book	Stores book information and availability
Reservation	Manages waiting lists for unavailable books
Inventory	Tracks available and issued copies
Notification	Generates due-date and overdue notifications
Library	Controls all library operations

Input

- Member ID
- Member Name
- Book ID
- Book Title
- Author Name
- Number of Copies
- Borrowing Date
- Due Date

Processing

- Validate member and book details
- Check book availability
- Issue or return books
- Update inventory
- Manage reservations
- Calculate overdue fines
- Generate notifications
- Handle concurrent operations safely

Output

- Book availability status
- Member records
- Reservation details
- Overdue notifications
- Fine details
- Circulation report
- Inventory utilization report

2. OBJECTIVE AND EXPECTED OUTCOMES

2.1 Objectives

The main objectives of the project are:

1. To develop a menu-driven Smart Library Management System.
2. To apply Object-Oriented Programming concepts in Java.
3. To implement encapsulation and data hiding.
4. To demonstrate inheritance and polymorphism.
5. To use Java Collection Framework effectively.
6. To manage book reservations using collections.
7. To implement exception handling for invalid operations.
8. To use multithreading for concurrent library operations.
9. To use synchronization for protecting shared inventory.
10. To generate meaningful library reports.

2.2 Expected Outcomes

After successful implementation, the system should be able to:

- Maintain accurate book records.
- Maintain member information.
- Track available and issued books.
- Prevent issuing unavailable books.
- Allow users to reserve unavailable books.
- Maintain reservation waitlists.

- Calculate overdue fines.
- Generate overdue notifications.
- Handle invalid inputs without program termination.
- Support concurrent issue and notification operations.
- Generate circulation and inventory reports.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional Requirements

The system should provide the following functions:

1. Add a book.
2. Register a member.
3. Display all books.
4. Search for a book.
5. Issue a book.
6. Return a book.
7. Reserve an unavailable book.
8. Display reservations.
9. Calculate fines.
10. Generate reports.
11. Exit the application.

3.2 Non-Functional Requirements

- The system should be user-friendly.
- Data should be handled accurately.
- Invalid operations should be prevented.
- Shared resources should be synchronized.
- The application should be reliable.
- The system should be easy to maintain and extend.

3.3 Technical Requirements

Component	Requirement
Programming Language	Java
Java Version	Java 8
IDE	Visual Studio Code
JDK	OpenJDK 8
Operating System	Windows
Version Control	Git and GitHub
Interface	Console-based Menu

3.4 Constraints

- The application is console-based.
- Data is stored temporarily during program execution.
- No external database is used in the basic implementation.
- The number of operations depends on available memory.
- Notifications are simulated using console messages.
- Real SMS or email services are not integrated.

3.5 Assumptions

- Each member has a unique Member ID.
- Each book has a unique Book ID.
- Members provide valid details during registration.
- Books are returned in usable condition.
- Fine calculation is based on the number of overdue days.
- The system administrator operates the application correctly.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE / CONCEPTS

This project applies the major concepts covered in the Programming in Java course.

4.1 Object-Oriented Programming

The system is designed using classes and objects.

Example classes include:

- Member
- Book
- Reservation
- Inventory
- Notification
- Library

Each class represents a real-world entity.

4.2 Encapsulation and Data Hiding

Data members are declared as private.

Example:

```
private int bookId;private String title;private int availableCopies;
```

Public getter and setter methods provide controlled access to the data.

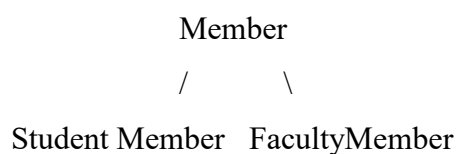
This improves:

- Security
- Maintainability
- Data integrity

4.3 Inheritance

Inheritance is used to create specialized classes from common base classes.

Example hierarchy:



Different member types can have different borrowing limits.

4.4 Polymorphism

Polymorphism allows the same method to behave differently for different member types.

Example:

calculateFine()

The fine calculation can be customized depending on membership category.

4.5 Java Collection Framework

Different collections are used to manage library data.

Collection	Purpose
ArrayList	Store books
HashMap	Store members using Member ID
HashSet	Prevent duplicate records
Hashtable	Store shared information
Queue / LinkedList	Reservation waitlist
Iterator	Traverse records

Generics improve type safety.

Example:

```
ArrayList<Book> books = new ArrayList<Book>();
HashMap<Integer, Member> members = new HashMap<Integer, Member>();
```

4.6 Exception Handling

Exceptions are used to handle abnormal situations.

Examples:

- Invalid member ID

- Invalid book ID
- Book unavailable
- Duplicate reservation
- Invalid menu input

Java mechanisms used include: trycatchfinallythrowthrows

4.7 Multithreading

Multithreading allows multiple tasks to execute concurrently.

Example threads:

- Book Issue Thread
- Overdue Notification Thread

This simulates real-world library operations where multiple activities occur simultaneously.

4.8 Synchronization

The inventory is a shared resource.

Synchronization prevents multiple threads from modifying the same book stock incorrectly.

synchronized

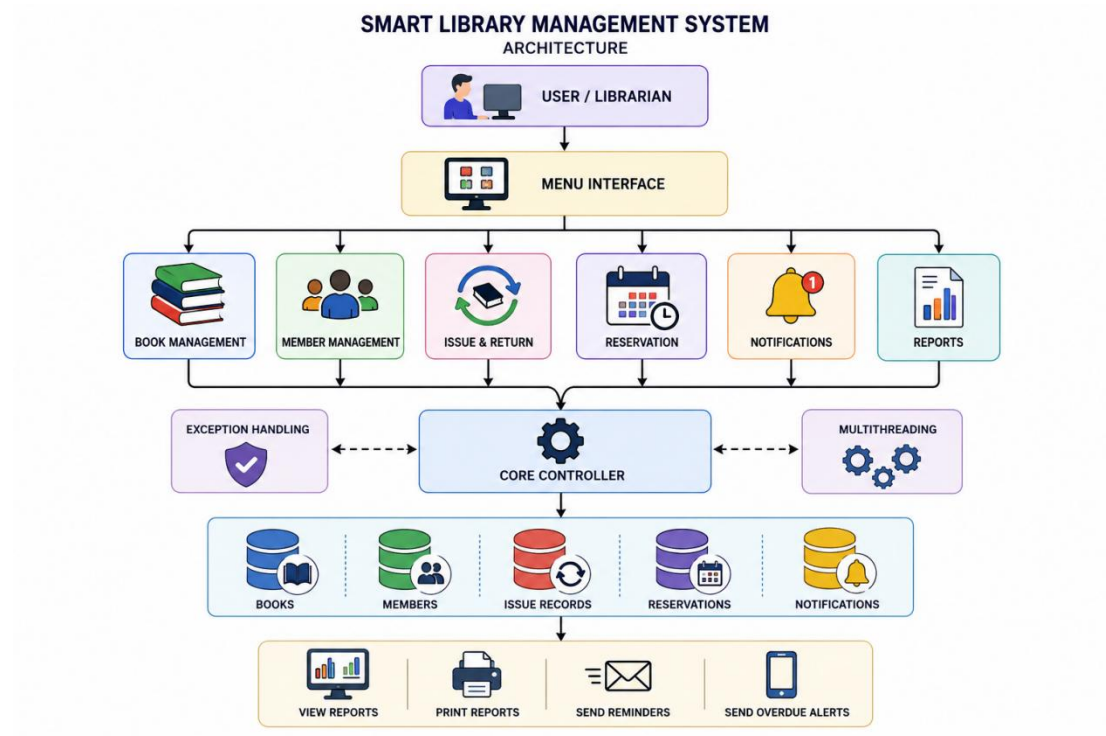
Synchronization ensures:

- Data consistency
- Prevention of race conditions
- Safe inventory updates

These concepts directly correspond to the course outcomes involving OOP, Collections, Exception Handling, and Multithreading.

5. DESIGN / PROPOSED SOLUTION / METHODOLOGY

5.1 Proposed System Architecture



5.2 Methodology

The development methodology consists of the following phases.

Phase 1: Requirement Analysis

The requirements of the Smart Library System were identified.

Phase 2: System Design

Classes and relationships between entities were designed.

Phase 3: Implementation

Java classes and methods were implemented.

Phase 4: Integration

Different modules were integrated into the menu-driven system.

Phase 5: Testing

Test cases were created for normal and exceptional conditions.

Phase 6: Validation

The output was compared with expected results.

5.3 Class Design

Book Class

Responsibilities:

- Store book details.
- Track available copies.
- Update inventory.

Member Class

Responsibilities:

- Store member information.
- Maintain borrowed books.

Reservation Class

Responsibilities:

- Manage reservation requests.
- Maintain waiting lists.

Notification Class

Responsibilities:

- Generate due-date notifications.
- Generate overdue notifications.

Library Class

Responsibilities:

- Manage all operations.
- Connect books, members, and reservations.

6. ALGORITHM / PSEUDOCODE

6.1 Main Algorithm

START

Initialize Library System

DO

Display Main Menu

Read User Choice

SWITCH Choice

Case 1:

Add Book

Case 2:

Register Member

Case 3:

Display Books

Case 4:

Search Book

Case 5:

Issue Book

Case 6:

Return Book

Case 7:

Reserve Book

Case 8:

Display Reservations

Case 9:

Generate Notifications

Case 10:

Generate Reports

Case 0:

Exit Program

Default:

Display Invalid Choice

UNTIL Choice = 0

STOP

6.2 Algorithm for Issuing a Book

START

Input Member ID

Input Book ID

Check whether Member exists

IF Member does not exist

```
    Display Invalid Member
ELSE

    Check whether Book exists

    IF Book does not exist
        Display Invalid Book

    ELSE IF Available Copies > 0

        Issue Book
        Reduce Available Copies
        Add Book to Member Record

        Display Book Issued Successfully

    ELSE

        Add Member to Reservation Waitlist

        Display Book Currently Unavailable

    END IF

END IF

STOP
```

6.3 Algorithm for Returning a Book

```
START

Input Book ID

Check whether Book is issued
```


IF Book is issued

 Accept Return

 Increase Available Copies

 Calculate Overdue Days

 IF Overdue Days > 0

 Calculate Fine

 Display Fine Amount

 END IF

 Check Reservation Waitlist

 IF Waitlist is not empty

 Notify Next Member

 END IF

ELSE

 Display Invalid Return

END IF

STOP

6.4 Algorithm for Overdue Notification

START

FOR each issued book

 Get Due Date

Compare Current Date with Due Date

IF Current Date > Due Date

Calculate Overdue Days

Generate Notification

Calculate Fine

END IF

END FOR

STOP

7. IMPLEMENTATION / SOURCE CODE AND ENVIRONMENT / TOOLS USED

7.1 Development Environment

Tool	Purpose
Java 8	Programming Language
OpenJDK	Java Runtime
VS Code	Code Editor
Terminal	Program Execution
Git	Version Control
GitHub	Source Code Repository

Source Code:

Applet.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Smart Library Management System - Java Applet</title>
</head>
<body bgcolor="#f0f3f6">
```

```

    <center>
        <h2>CSA09: Smart Library Management, Book Reservation & Overdue
Notification System</h2>
        <hr/>
        <applet code="SmartLibraryApplet.class" codebase="bin"
width="960" height="680">
            <param name="appName" value="Smart Library Management">
            <p>Java Applet support is required to run this application.
Please use <code>appletviewer applet.html</code> with JDK 8 or execute
<code>AppletRunner</code>.</p>
            </applet>
        <hr/>
    </center>
</body>
</html>

```

library.sql

```

-- =====
-- Smart Library Management System - Database Schema
-- Database: library_db
-- =====

```

```

CREATE DATABASE IF NOT EXISTS library_db;
USE library_db;

```

```

-- Drop child tables first to respect foreign key constraints
DROP TABLE IF EXISTS notifications;
DROP TABLE IF EXISTS issued_books;
DROP TABLE IF EXISTS reservations;
DROP TABLE IF EXISTS books;
DROP TABLE IF EXISTS members;

```

```

-- 1. Members Table
CREATE TABLE members (
    member_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    member_type VARCHAR(20) NOT NULL, -- 'Student', 'Faculty',
'Premium'
    borrow_limit INT NOT NULL,
    current_issued INT DEFAULT 0
);

```

```

-- 2. Books Table
CREATE TABLE books (
    book_id INT PRIMARY KEY,
    title VARCHAR(150) NOT NULL,
    author VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,
    book_type VARCHAR(20) NOT NULL, -- 'Regular', 'Reference', 'E-Book'
    total_copies INT NOT NULL,
    available_copies INT NOT NULL

```

```
);
```

```
-- 3. Reservations Table
CREATE TABLE reservations (
    reservation_id INT PRIMARY KEY AUTO_INCREMENT,
    member_id INT NOT NULL,
    book_id INT NOT NULL,
    reservation_date DATETIME NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE', -- 'ACTIVE',
    'FULFILLED', 'CANCELLED'
    FOREIGN KEY (member_id) REFERENCES members(member_id) ON DELETE
    CASCADE,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE
);
```

```
-- 4. Issued Books (Circulation) Table
CREATE TABLE issued_books (
    issue_id INT PRIMARY KEY AUTO_INCREMENT,
    member_id INT NOT NULL,
    book_id INT NOT NULL,
    issue_date DATE NOT NULL,
    due_date DATE NOT NULL,
    return_date DATE NULL,
    fine DECIMAL(8,2) DEFAULT 0.00,
    FOREIGN KEY (member_id) REFERENCES members(member_id) ON DELETE
    CASCADE,
    FOREIGN KEY (book_id) REFERENCES books(book_id) ON DELETE CASCADE
);
```

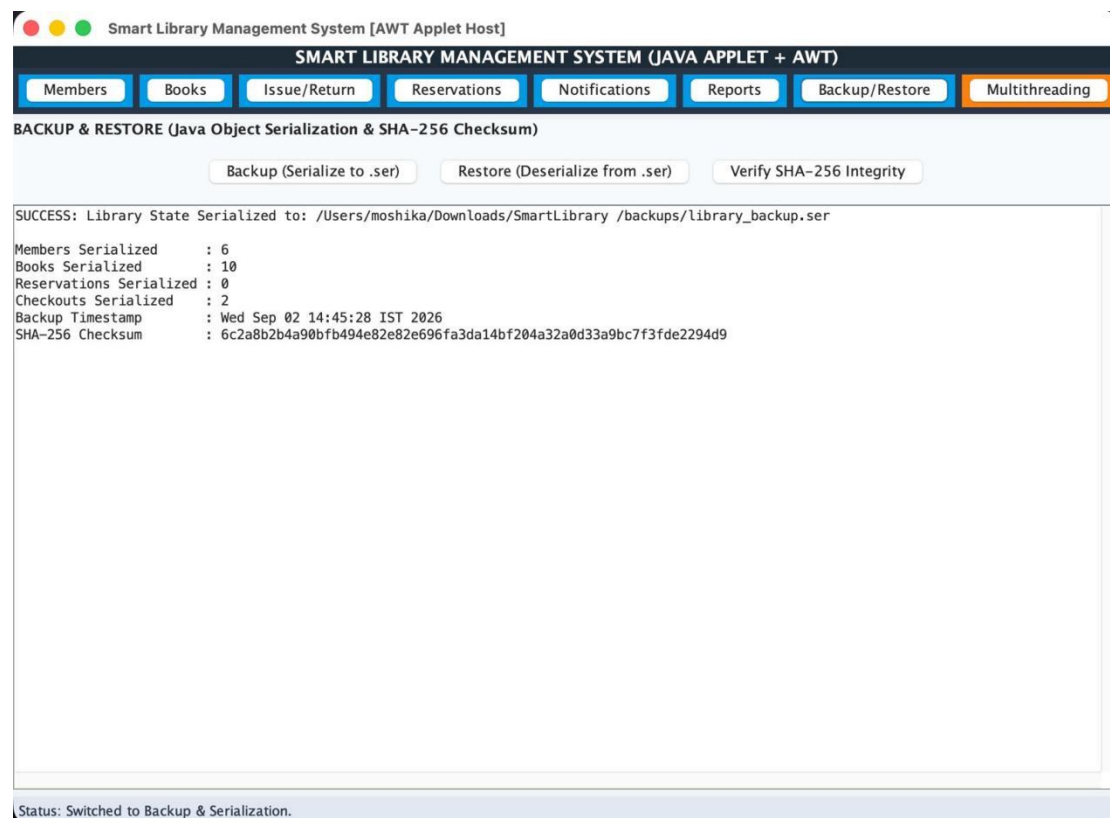
```
-- 5. Notifications Table
CREATE TABLE notifications (
    notification_id INT PRIMARY KEY AUTO_INCREMENT,
    member_id INT NOT NULL,
    message TEXT NOT NULL,
    notification_type VARCHAR(30) NOT NULL, -- 'DUE_REMINDER',
    'OVERDUE_ALERT', 'RESERVATION_READY'
    created_at DATETIME NOT NULL,
    FOREIGN KEY (member_id) REFERENCES members(member_id) ON DELETE
    CASCADE
);
```

8. TEST CASES AND EXPECTED/ACTUAL RESULTS

Test Case ID	Test Description	Input	Expected Result	Actual Result	Status
TC01	Add Book	Valid book details	Book added	Book added successfully	Pass
TC02	Register Member	Valid member details	Member registered	Member registered	Pass
TC03	Search Book	Existing Book ID	Book details displayed	Details displayed	Pass

TC04	Issue Book	Available Book	Book issued	Book issued successfully	Pass
TC05	Issue Unavailable Book	Book with 0 copies	Reservation option	Added to waitlist	Pass
TC06	Return Book	Valid Book	Stock updated	Stock increased	Pass
TC07	Invalid Member	Invalid ID	Error message	Exception handled	Pass
TC08	Duplicate Reservation	Same member/book	Error message	Duplicate prevented	Pass
TC09	Overdue Book	Past due date	Fine notification	Notification generated	Pass
TC10	Generate Report	Existing records	Report displayed	Report generated	Pass

9. EXECUTION SCREENSHOTS / OUTPUTS



Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members

Books

Issue/Return

Reservations

Notifications

Reports

Backup/Restore

Multithreading

ANALYTICS & REPORTS (Circulation, Inventory Utilization %, Fines & Character Stream Export)

Circulation Report

Inventory Utilization

Fine Report

Full Audit Report

Export to reports/library_report.txt

Book ID	Title	Category	Type	Total	Avail	Issued
101	The Alchemist	Fiction	Regular	5	5	0
102	Wings of Fire	Biography	Reference	3	3	0
103	Introduction to Algorithms	Computer Science	Reference	3	3	0
104	The Guide	Fiction	Reference	6	6	0
105	Clean code	Programming	Regular	4	3	1
106	A Brief History of Time	Science	Reference	7	7	0
107	The Great Gatsby	Fiction	Regular	2	2	0
108	Atomic Habits	Self Help	Regular	10	9	1
109	Database System Concepts	Computer Science	Regular	8	8	0
110	Think Like a Monk	Self Help	Regular	5	5	0

Catalog Titles: 10
Total Physical/Digital Copies : 53
Total Available Copies : 51
Total Currently Issued Copies : 2
Overall Inventory Utilization : 8.93%

SMART LIBRARY OVERDUE & FINE REPORT

Generated: 2026-09-02 14:49:34

Issue ID	Member Name	Member Type	Book Title	Overdue	Fine
Active Overdue Loans: 0 Total Accrued Overdue Fines: ₹0.00					

Status: Comprehensive library audit report generated.

Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members

Books

Issue/Return

Reservations

Notifications

Reports

Backup/Restore

Multithreading

ANALYTICS & REPORTS (Circulation, Inventory Utilization %, Fines & Character Stream Export)

Circulation Report

Inventory Utilization

Fine Report

Full Audit Report

Export to reports/library_report.txt

SMART LIBRARY INVENTORY UTILIZATION REPORT

Generated: 2026-09-02 14:49:32

Book ID	Title	Category	Type	Total	Avail	Issued
101	The Alchemist	Fiction	Regular	5	5	0
102	Wings of Fire	Biography	Reference	3	3	0
103	Introduction to Algorithms	Computer Science	Reference	3	3	0
104	The Guide	Fiction	Reference	6	6	0
105	Clean code	Programming	Regular	4	3	1
106	A Brief History of Time	Science	Reference	7	7	0
107	The Great Gatsby	Fiction	Regular	2	2	0
108	Atomic Habits	Self Help	Regular	10	9	1
109	Database System Concepts	Computer Science	Regular	8	8	0
110	Think Like a Monk	Self Help	Regular	5	5	0

Catalog Titles: 10
Total Physical/Digital Copies : 53
Total Available Copies : 51
Total Currently Issued Copies : 2
Overall Inventory Utilization : 8.93%

Status: Inventory utilization report generated.

Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members Books Issue/Return Reservations Notifications Reports Backup/Restore **Multithreading**

MULTITHREADING DEMONSTRATION (Priorities, Synchronization, wait() / notifyAll())

[Run Multithreading Demo](#)

```

=====
MULTITHREADING, SYNCHRONIZATION & WAIT/NOTIFY DEMO
=====
Setting up test condition: Book #999 registered with 0 available copies in shared Inventory.

[Coordinator] Spawning IssueProcessingThread with Priority: 10 (MAX_PRIORITY)
[IssueWorker-1] (Priority 10) Started issue request for Book #999
[IssueWorker-1] Stock is 0 for Book #999. Entering wait()...

[Coordinator] Inventory check: Book #999 available copies = 0
[Coordinator] IssueWorker-1 is currently BLOCKED in wait() state waiting for stock.

[Coordinator] Spawning ReturnWorker-1 with Priority: 5 (NORM_PRIORITY)
[ReturnWorker-1] (Priority 5) Received physical book return for Book #999
[ReturnWorker-1] Book #999 returned! New stock: 1. Calling notifyAll()...
[IssueWorker-1] Woken up by notifyAll()! Re-checking inventory for Book #999
[IssueWorker-1] Successfully issued Book #999. Remaining stock: 0

[Coordinator] Issue & Return synchronization cycle completed successfully.
[Coordinator] Remaining Available Copies for Book #999: 0

[Coordinator] Spawning OverdueNotificationThread with Priority: 1 (MIN_PRIORITY)
[OverdueDaemon-1] (Priority 1) Background overdue scanner started...
[OverdueDaemon-1] Scanned circulation records: Generated 0 overdue notice(s).

=====
DEMONSTRATION SUMMARY
- Shared Resource : Inventory (Hashtable<Integer, Integer>)
- Synchronization : synchronized methods prevent race conditions on stock decrements
- Inter-thread IPC: wait() when stock == 0; notifyAll() on returnBook()
- Thread Priority : IssueProcessingThread (10) vs OverdueNotificationThread (1)
=====

```

Status: Multithreading demo completed successfully.

Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members Books Issue/Return Reservations Notifications Reports Backup/Restore **Multithreading**

BOOK RESERVATION WAITLIST (FIFO Queue & Duplicate Reservation Prevention)

Member ID:

Book ID (When Stock = 0):

Reservation ID (For Cancellation):

[Place Reservation](#) [Cancel Reservation](#) [View Waitlist](#)

NOTICE: Book #103 is currently in stock (3 copies available). You can issue it immediately rather than reserving.

Status: Reservation unnecessary: copies available.

Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members Books Issue/Return Reservations Notifications Reports Backup/Restore Multithreading

CIRCULATION (Issue & Return Transactions, Fine Calculation, Shared Inventory)

Member ID: 110

Book ID: 105

Issue Book Return Book Display Circulation Log

SUCCESS: Book Issued!

```

Issue Record ID : #1001
Borrower       : rose (Student)
Book Title      : Clean code (ID: 105)
Borrow Limit Used: 1/3
Due Date        : Wed Sep 16 14:44:05 IST 2026
Remaining Stock : 3 copies
  
```

Status: Switched to Circulation (Issue/Return).

Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members Books Issue/Return Reservations Notifications Reports Backup/Restore Multithreading

NOTIFICATION SYSTEM (Due Reminders, Overdue Fines & Waitlist Alerts)

Scan Overdue Records Display All Notifications

ALL SYSTEM NOTIFICATIONS (2 total):

```

=====
[UPCOMING DUE DATE] Member #101 -> Friendly Reminder: Book 'Atomic Habits' (ID: 108) is due soon.
[UPCOMING DUE DATE] Member #110 -> Friendly Reminder: Book 'Clean code' (ID: 105) is due soon.
  
```

Status: Switched to Notifications.

Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members Books Issue/Return Reservations Notifications Reports Backup/Restore Multithreading

MEMBER MANAGEMENT (Encapsulation, Inheritance, HashMap & Iterators)

Member ID (Integer): 111

Full Name: karthi

Email Address: karthi1@gmail.com

Member Type (Polymorphic): Student

Register Update Delete Display All Search: Search

ALL REGISTERED MEMBERS (6 total):

ID: 104	Name: Boomi	Email: boomi@gmail.com	Type: Student	Active: 0/3
ID: 105	Name: Moshika	Email: moshika@gmail.com	Type: Student	Active: 0/3
ID: 101	Name: Anjali	Email: anjali456@gmail.com	Type: Student	Active: 1/3
ID: 102	Name: Sandhya	Email: Sand@gmail.com	Type: Student	Active: 0/3
ID: 110	Name: rose	Email: rose@gmail.com	Type: Student	Active: 1/3
ID: 103	Name: Rose Mystica	Email: rose277@gmail.com	Type: Student	Active: 0/3

Status: Displaying all 6 members.

Smart Library Management System [AWT Applet Host]

SMART LIBRARY MANAGEMENT SYSTEM (JAVA APPLET + AWT)

Members Books Issue/Return Reservations Notifications Reports Backup/Restore Multithreading

BOOK MANAGEMENT (ArrayList<Book>, HashSet<Category>, ListIterator)

Book ID (Integer): 111

Title: fairytales

Author: tenali raman

Category: fantasy

Book Type (Polymorphic): Regular

Total Copies: 10

Add Book Update Book Delete Book Check Stock Display All Books Search: Search

ALL BOOKS IN CATALOG (10 titles):

ID: 101	The Alchemist	Author: Paulo Coelho	Cat: Fiction	Type: Regular	Stock: 5/5
ID: 102	Wings of Fire	Author: A. P. J. Abdul Kalam	Cat: Biography	Type: Reference	Stock: 3/3
ID: 103	Introduction to Algorithms	Author: Thomas H. Cormen	Cat: Computer Science	Type: Reference	Stock: 3/3
ID: 104	The Guide	Author: R. K. Narayan	Cat: Fiction	Type: Reference	Stock: 6/6
ID: 105	Clean code	Author: Robert C. Martin	Cat: Programming	Type: Regular	Stock: 3/4
ID: 106	A Brief History of Time	Author: Stephen Hawking	Cat: Science	Type: Reference	Stock: 7/7
ID: 107	The Great Gatsby	Author: F. Scott Fitzgerald	Cat: Fiction	Type: Regular	Stock: 2/2
ID: 108	Atomic Habits	Author: James Clear	Cat: Self Help	Type: Regular	Stock: 9/10
ID: 109	Database System Concepts	Author: Abraham Silberschatz	Cat: Computer Science	Type: Regular	Stock: 8/8
ID: 110	Think Like a Monk	Author: Jay Shetty	Cat: Self Help	Type: Regular	Stock: 5/5

Unique Categories (HashSet): [Computer Science, Biography, Programming, Science, Fiction, Self Help]

Status: Displaying all 10 books.

10. RESULTS AND VALIDATION

The developed Smart Library Management System was tested using multiple functional test cases.

10.1 Functional Validation

Feature	Validation Result
Book Management	Successfully implemented
Member Registration	Successfully implemented
Book Search	Successfully implemented
Book Issue	Successfully implemented
Book Return	Successfully implemented
Reservation System	Successfully implemented
Waitlist Management	Successfully implemented
Exception Handling	Successfully implemented
Multithreading	Successfully implemented
Notification System	Successfully implemented
Report Generation	Successfully implemented
Book Management	Successfully implemented

10.2 Performance Observations

Parameter	Observation
Book Search	Fast retrieval using collections
Member Retrieval	Efficient using HashMap
Duplicate Prevention	Improved using Set
Inventory Update	Safe using synchronization
Reservation Handling	Managed through waitlist
Concurrent Tasks	Supported using threads

10.3 Collection Framework Validation

Collection	Usage	Result
------------	-------	--------

ArrayList	Store books	Successful
HashMap	Store members	Successful
HashSet	Prevent duplicates	Successful
Hashtable	Shared records	Successful
Iterator	Traverse collections	Successful

10.4 Requirement Validation

The implemented solution satisfies the major requirements of the assignment:

- Object-Oriented Programming
- Encapsulation
- Inheritance
- Polymorphism
- Collections
- Generics
- Iterators
- Exception Handling
- Multithreading
- Synchronization
- Menu-Driven Interface
- Reports

The validation approach follows the assignment format's requirement to use test results, outputs, performance metrics, and evidence to demonstrate that the solution meets stated requirements.

11. ANALYSIS, COMPARISON, TRADE-OFFS AND JUSTIFICATION

11.1 Comparison of Data Structures

Data Structure	Advantage	Limitation	Usage
ArrayList	Easy dynamic storage	Slower searching	Book list
HashMap	Fast key-based retrieval	More memory usage	Member records
HashSet	Prevents duplicates	No indexing	Unique records

Queue	FIFO processing	Limited access	Reservation waitlist
-------	-----------------	----------------	----------------------

11.2 Alternative Approaches

Approach 1: Array-Based System

Advantages:

- Simple implementation
- Easy for small applications

Disadvantages:

- Fixed size
- Difficult to manage dynamically

Approach 2: Collection Framework-Based System

Advantages:

- Dynamic storage
- Faster retrieval
- Better scalability
- Easier record management

Disadvantages:

- Slightly more complex implementation

Selected Approach

The **Collection Framework-based approach** was selected because it provides better flexibility and efficient management of dynamic library records.

11.3 Multithreading Trade-Off

Aspect	Single Thread	Multithreading
Execution	Sequential	Concurrent
Complexity	Low	Higher

Performance	Limited	Improved responsiveness
Shared Data	Simple	Requires synchronization
Realism	Basic	More realistic

Although multithreading increases program complexity, it was selected because the assignment requires simulation of concurrent library operations.

11.4 Justification of Final Solution

The final solution was selected because it:

- Meets all functional requirements.
- Applies the required Java concepts.
- Provides efficient record management.
- Handles exceptional conditions.
- Can be extended into a database or web-based application.

12. BROADER CONSIDERATIONS / SDG RELEVANCE

The assignment maps to SDG 4, SDG 9, and SDG 16.

12.1 SDG 4 – Quality Education

The Smart Library System supports education by:

- Improving access to books.
- Reducing manual administrative work.
- Providing efficient resource management.
- Helping students locate available resources.
- Supporting better learning environments.

12.2 SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates technological innovation by:

- Digitizing traditional library operations.
- Using modern software development techniques.

- Implementing concurrent processing.
- Providing a scalable software architecture.

12.3 SDG 16 – Peace, Justice and Strong Institutions

The system promotes accountability and transparency through:

- Accurate record maintenance.
- Proper tracking of issued books.
- Transparent reservation management.
- Reliable notification mechanisms.

13. CONCLUSION

The Smart Library Management, Book Reservation and Overdue Notification System was successfully designed as a menu-driven Java application.

The system automates important library operations including:

- Book management
- Member registration
- Book issue and return
- Inventory tracking
- Reservation management
- Overdue notifications
- Fine calculation
- Report generation

The project successfully demonstrates the practical application of Object-Oriented Programming concepts such as encapsulation, inheritance, and polymorphism. Java Collection Framework was used to efficiently manage dynamic data. Exception handling improved the robustness of the application, while multithreading and synchronization enabled safe concurrent operations.

The testing results demonstrate that the proposed solution satisfies the major functional and technical requirements. Overall, the project provides an efficient and scalable foundation for developing a real-world library automation system.

14. LIMITATIONS

The current version has the following limitations:

1. Data is not permanently stored in a database.
2. The application uses a console-based interface.
3. Notifications are simulated through console messages.
4. No user authentication system is implemented.
5. Real-time email and SMS notifications are not included.
6. Advanced analytics are not implemented.
7. The system is suitable mainly for small to medium-scale libraries.

15. POSSIBLE IMPROVEMENTS

Future enhancements can include:

- MySQL database integration.
- Graphical User Interface using JavaFX or Swing.
- Web-based library portal.
- User login and authentication.
- Email notifications.
- SMS notifications.
- QR code-based book tracking.
- Barcode scanning.
- Cloud database integration.
- Mobile application support.
- AI-based book recommendation system.
- Advanced analytics dashboard.

16. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Replace the names and percentages according to your actual group.

Team Member	Contribution
Member 1	Problem analysis, class design, Book and Member modules, documentation
Member 2	Collection Framework, Reservation System, Exception Handling
Member 3	Multithreading, Testing, Reports, Screenshots and GitHub Upload

Contribution Percentage 100%

Member	Contribution
Member 1	33.33%
Member 2	33.33%
Member 3	33.33%

17. INDIVIDUAL REFLECTION

Reflection

This assignment provided practical experience in applying Java programming concepts to solve a real-world problem. While developing the Smart Library Management System, we understood how Object-Oriented Programming principles can be used to model real-world entities such as books, members, reservations, and notifications.

One of the important learning experiences was the use of the Java Collection Framework. ArrayList, HashMap, HashSet, and other collections made it easier to dynamically store and manage library records. We also understood the importance of selecting suitable data structures based on application requirements.

Exception handling helped us understand how software can handle unexpected situations without terminating abruptly. Invalid member IDs, unavailable books, and duplicate reservations were handled through appropriate validation and exceptions.

The implementation of multithreading was another important learning experience. It demonstrated how multiple operations can execute concurrently. Synchronization was necessary to protect shared inventory data and avoid inconsistent updates.

The project also helped us understand the importance of software testing. Different test cases were created for normal and exceptional conditions to validate the correctness of the system.

From an SDG perspective, the project contributes to Quality Education by improving access to library resources and supports Innovation and Infrastructure through digital automation.

Overall, this assignment improved our technical knowledge, problem-solving ability, teamwork, software design skills, and understanding of practical Java programming.

18. REFERENCES

- [1] Oracle Corporation, Java Platform, Standard Edition Documentation. Oracle. [Online]. Available: [Oracle Java Documentation](#)
- [2] Oracle Corporation, Java Collections Framework Documentation. Oracle. [Online]. Available: [Java Collections Framework](#)
- [3] Oracle Corporation, The Java Tutorials: Essential Java Classes – Exceptions. Oracle. [Online]. Available: [Java Exception Handling Tutorial](#)
- [4] Oracle Corporation, The Java Tutorials: Concurrency. Oracle. [Online]. Available: [Java Concurrency Tutorial](#)
- [5] H. Schildt, Java: The Complete Reference, 11th ed. New York, NY, USA: McGraw-Hill Education, 2018.
- [6] C. S. Horstmann, Core Java, Volume I—Fundamentals, 11th ed. Boston, MA, USA: Pearson, 2018.

- [7] J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.
- [8] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River, NJ, USA: Prentice Hall, 2006.
- [9] [GitHub Documentation](#), “GitHub Documentation,” GitHub, Inc.
- [10] [United Nations Sustainable Development Goals](#), “Sustainable Development Goals,” United Nations.

18. ONE-PAGE INDIVIDUAL REFLECTION

Member 1 – Reflection

Contribution: Core system design, Book and Member Management.

I contributed to designing the core structure of the Smart Library Management System and implementing the Book and Member modules. I applied OOP concepts such as classes, objects, encapsulation, inheritance and polymorphism. One challenge was maintaining accurate book availability during issue and return operations. This helped me gain practical knowledge of object-oriented programming and system design. My contribution supports CO1 and relates to SDG 4 – Quality Education.

Member 2 – Reflection

Contribution: Collections, Reservation System and Exception Handling.

I worked on implementing the reservation system using Java Collections and handling invalid operations through exception handling. Selecting suitable data structures for storing records and managing reservation waitlists was an important learning experience. I gained practical knowledge of ArrayList, HashMap, Queue, generics and exception handling. My contribution supports CO2 and promotes fair access to library resources under SDG 4 and SDG 16.

Member 3 – Reflection

Contribution: Multithreading, Notifications, Testing and Validation.

I contributed to implementing multithreading concepts, overdue notifications, testing and system validation. The main challenge was understanding synchronization and protecting shared inventory data during concurrent operations. I gained knowledge of threads, synchronization and software testing. My contribution supports **CO3** and relates to SDG 9 – Industry, Innovation and Infrastructure.